

TASK4 复刻传奇：海龟交易法则实战演练

在量化交易领域，交易策略是实现盈利的核心。本任务的主要目标是运用 Python 实现海龟交易策略，掌握其独特的高低点通道、平均真实波幅 (ATR) 以及止损条件等关键概念。同时计算最大回撤 (MDD)、夏普比率 (Sharpe Ratio)、累计回报 (Cumulative Return) 等核心指标，全面评估海龟策略的性能，为后续的交易策略开发提供宝贵的经验和参考。

任务 1. 了解海龟策略，给出海龟策略的核心思想，有哪些关键优势？

海龟策略 (Turtle Trading Strategy) 是一种经典的趋势跟踪交易策略，由美国交易员 Richard Dennis 和 William Eckhardt 于 20 世纪 80 年代提出。该策略最初来源于海龟交易员实验，即通过一套标准化、可复制的交易规则培养没有交易经验的新手交易员，从而验证优秀交易员是否能够通过系统化训练培养出来。海龟策略的核心思想是“顺势而为、截断亏损、让利润奔跑”，认为市场的大部分收益来源于少数几次持续性的趋势行情，因此交易者无需预测市场未来走势，而是通过明确的规则捕捉已经形成的趋势，并在趋势结束前持续持有头寸。整个策略完全依赖客观的价格数据进行交易决策，包括入场、加仓、止损和离场等过程均具有固定规则，从而避免了人为主观判断带来的情绪干扰。

海龟策略的**核心思想**可以概括为突破入场、趋势持有、风险控制和动态加仓。在入场阶段，当价格突破过去一段时间内的最高价或最低价时，策略认为新的趋势已经形成，因此顺势建立多头或空头仓位；在持仓阶段，只要价格仍然保持趋势运行，策略便持续持有，而不会因为短期回调而轻易离场；与此同时，策略利用平均真实波幅 (Average True Range, ATR) 衡量市场波动程度，根据波动大小动态调整仓位规模和止损位置，使不同波动水平下承担的风险保持一致；若市场继续沿有利方向运行，则按照固定间隔逐步加仓，以扩大趋势行情中的收益，而一旦价格触发止损或跌破（突破）指定周期低点（高点），则立即退出市场。整个策略遵循“让盈利不断增长，而将亏损限制在较小范围”的原则，因此属于典型的风险收益非对称交易策略。

与均线交叉策略相比，海龟策略**第一个优势**在于能够更快地识别新趋势并提高风险控制能力。均线交叉策略依赖移动平均线之间的交叉产生交易信号，而移动平均线本质上属于历史价格的平滑结果，因此具有一定的滞后性，往往需要趋势已经运行一段时间之后才发出买卖信号，容易错过趋势初期最具盈利能力的阶段。相比之下，海龟策略直接利用价格突破历史高低点作为入场依据，对市场变化更加敏感，因此通常能够更早进入趋势行情。**第二个优**

势在于海龟策略采用 ATR 进行仓位管理和止损控制，而均线交叉策略通常仅依据均线交叉决定买卖，缺乏统一的风险管理机制。当市场波动较大时，ATR 会自动扩大止损范围并减少仓位；当市场波动较小时，则适当提高仓位规模，从而使每笔交易承担相近的风险。这种动态风险控制机制能够有效降低市场剧烈波动带来的损失，而均线交叉策略在不同波动环境下容易出现风险暴露不一致的问题。**第三个优势**在于海龟策略较强的盈利扩展能力。均线交叉策略一般在均线反向交叉时一次性平仓，较少考虑趋势中的持续加仓；而海龟策略在趋势确认后允许按照固定 ATR 间隔逐步增加仓位，使盈利头寸不断扩大，从而充分利用少数持续时间较长的大趋势获取超额收益。因此，在趋势明显的市场环境下，海龟策略通常能够获得比传统均线交叉策略更高的收益。然而，其代价是在震荡行情中容易频繁触发突破和止损，交易次数较多，因此通常需要结合市场环境进行合理应用。

任务 2. 对海龟策略中的高低点通道、平均真实波幅（ATR）以及止损条件等概念做出解释

1. 高低点通道 Price Channel

高低点通道（Price Channel）是海龟策略中用于判断趋势突破的重要工具，其定义为过去连续若干个交易日内的最高价和最低价所构成的价格区间。例如，当采用 20 日通道时，上轨表示最近 20 个交易日中的最高价格，下轨表示最近 20 个交易日中的最低价格。海龟策略认为，当价格向上突破通道上轨时，意味着市场可能开始进入新的上涨趋势，因此产生买入信号；反之，当价格跌破通道下轨时，则认为新的下跌趋势已经形成，从而产生卖出或做空信号。

2. 平均真实波幅 ATR

平均真实波幅（Average True Range, ATR）是由 J. Welles Wilder 提出的一项波动率指标，用于衡量市场价格波动的剧烈程度。ATR 首先计算每日的真实波幅（True Range, TR），其取值为以下三项中的最大值：当日最高价与最低价之差、当日最高价与前一日收盘价之差的绝对值，以及当日最低价与前一日收盘价之差的绝对值。随后，再对连续若干个交易日的真实波幅取平均，即可得到 ATR 值。ATR 越大，说明市场波动越剧烈；ATR 越小，则表示市场波动较为平稳。

在海龟策略中，ATR 主要用于风险管理，而不是生成交易信号。首先，ATR 决定每次建仓的仓位大小，波动越大的市场采用越小的仓位，波动越小则允许建立更大的仓位，从而使每笔交易承担相近的风险；其次，ATR 还用于确定止损距离和加仓间隔，例如经典海龟

策略通常将止损设置为距离建仓价格 2 倍 ATR 的位置, 并按照每上涨 (或下跌) 0.5 倍 ATR 逐步增加仓位。

3. 止损条件 Stop-loss Rule

止损条件是海龟策略进行风险控制的重要组成部分, 其目的是在市场走势与预期相反时及时退出交易, 防止亏损进一步扩大。经典海龟策略通常采用 ATR 止损机制, 即当价格相对于建仓价格反向波动达到 2 倍 ATR 时立即平仓。例如, 若建立多头仓位后, 价格下跌超过 2ATR, 则执行止损; 若建立空头仓位后, 价格上涨超过 2ATR, 则同样止损离场。由于 ATR 能够反映当前市场波动水平, 因此这种止损方式相比固定百分比止损更加灵活, 在波动较大的市场中能够避免因正常价格波动而频繁止损, 在波动较小时又能够及时控制风险。此外, 海龟策略还设置趋势退出规则, 即当价格跌破 (或突破) 较短周期通道时, 也会主动结束持仓, 以确保能够及时退出已经结束的趋势行情。

4. 金字塔加仓 Pyramiding

金字塔加仓 (Pyramiding) 是海龟策略区别于普通突破策略的一项重要机制。当市场价格持续朝有利方向运行时, 策略不会一直保持初始仓位, 而是在价格每继续上涨 (或下跌) 约 0.5 倍 ATR 后逐步增加新的仓位, 直至达到预设的最大持仓数量。由于新增仓位建立在已有盈利基础之上, 因此整体风险并不会随着仓位增加而线性增长, 相反能够有效放大趋势行情中的累计收益。当趋势结束时, 所有仓位按照统一规则退出, 从而实现“盈利不断累积、亏损及时控制”的交易思想。

任务 3. python 编程实现以下内容:

- 1) 设定高低价格通道的周期, 如 20, 计算高低价格通道
- 2) 计算 ATR 数值
- 3) 计算买入卖出的交易信号
- 4) 根据计算结果绘制对应可视化图形, 包括股价, 高低价格通道, 交易信号, 标记买入信号, 卖出信号
- 5) 对策略进行模拟交易和回测, 计算以上量化指标

```
def run_turtle_backtest(  
    df: pd.DataFrame,  
    entry_period: int = DEFAULT_ENTRY_PERIOD,
```

```

exit_period: int = DEFAULT_EXIT_PERIOD,
atr_period: int = DEFAULT_ATR_PERIOD,
add_step: float = DEFAULT_ADD_STEP,
initial_capital: float = DEFAULT_INITIAL_CAPITAL,
commission_rate: float = 0.0,
start_date: str | None = None,
end_date: str | None = None,
) -> dict:

```

"""

运行海龟交易法则回测。

参数

df : pd.DataFrame 原始日线数据 trade_date, open, high, low, close 列
entry_period : int - 入场通道周期（唐奇安上轨，默认 20）
exit_period : int - 出场通道周期（唐奇安下轨，默认 10）
atr_period : int - ATR 周期（默认 14）
add_step : float - 加仓步长（几倍 ATR，默认 0.5）
initial_capital : float - 初始资金
commission_rate : float - 手续费率（0.0003 = 万分之三）
start_date, end_date : str | None - 回测日期范围

返回

dict - data（图表数据） + metrics（绩效指标）

"""

数据准备

```

df = df.copy()
if "trade_date" in df.columns:
    if not pd.api.types.is_datetime64_any_dtype(df["trade_date"]):
        df["trade_date"] =
pd.to_datetime(df["trade_date"].astype(str), format="%Y%m%d")
df = df.sort_values("trade_date").reset_index(drop=True)

# 日期过滤
if start_date is not None:
    start_dt = pd.to_datetime(start_date)
    df = df[df["trade_date"] >= start_dt]

```

```

if end_date is not None:
    end_dt = pd.to_datetime(end_date)
    df = df[df["trade_date"] <= end_dt]

df = df.reset_index(drop=True)
n = len(df)
if n == 0: raise ValueError("指定的日期范围内无交易数据")

max_period = max(entry_period, exit_period, atr_period)
if n < max_period + 2:
    raise ValueError(
        f"数据不足：回测区间仅 {n} 个交易日，"
        f"至少需要 {max_period + 2} 天"
    )

dates = df["trade_date"]
close = df["close"]
high = df["high"]
low = df["low"]

# 计算指标
atr = compute_atr(high, low, close, atr_period)
entry_upper, exit_lower = compute_donchian(high, low, entry_period,
exit_period)

# 模拟交易
buy_factor = 1.0 + commission_rate
sell_factor = 1.0 - commission_rate
stop_multiple = STOP_MULTIPLE
max_units = MAX_UNITS

cash = float(initial_capital)
total_shares = 0.0
unit_prices = [] # 每单位入场价
last_add_price = 0.0
portfolio_values = np.zeros(n)
trades = []

```

```

# 信号记录
buy_signals = [] # 首次入场
add_signals = [] # 加仓
sell_signals = [] # 通道出场
stop_signals = [] # 止损出场

# 止损线（用于绘图）
stop_line = np.full(n, np.nan)

for i in range(n):
    price_c = float(close.iloc[i])
    atr_val = float(atr.iloc[i]) if pd.notna(atr.iloc[i]) else None
    date_str = str(dates.iloc[i].date())

    # 计算单位头寸（基于当前 ATR 和初始资金）
    if atr_val and atr_val > 0:
        unit_shares = (initial_capital * RISK_PER_UNIT) / atr_val
    else: unit_shares = 0.0

    # 当前持仓状态
    holding = total_shares > 0
    num_units = len(unit_prices)

    # 入场信号：突破上轨
    upper = float(entry_upper.iloc[i]) if
pd.notna(entry_upper.iloc[i]) else None
    if not holding and upper is not None and price_c > upper and
unit_shares > 0:
        cost = unit_shares * price_c * buy_factor
        if cost <= cash:
            cash -= cost
            total_shares += unit_shares
            unit_prices.append(price_c)
            last_add_price = price_c
            buy_signals.append({"date": date_str, "price":
round(price_c, 2)})

    # 加仓信号：价格上涨 add_step × ATR

```

```

elif (holding and num_units < max_units
      and atr_val and atr_val > 0
      and last_add_price > 0
      and price_c >= last_add_price + add_step * atr_val
      and unit_shares > 0):
    cost = unit_shares * price_c * buy_factor
    if cost <= cash:
        cash -= cost
        total_shares += unit_shares
        unit_prices.append(price_c)
        last_add_price = price_c
        add_signals.append({"date": date_str, "price":
round(price_c, 2)})

# 出场 / 止损
if holding and num_units > 0:
    avg_entry = sum(unit_prices) / num_units
    stop_price = avg_entry - stop_multiple * atr_val if atr_val else
avg_entry * 0.85

# 记录止损线
stop_line[i] = stop_price if atr_val else np.nan

should_exit = False
exit_type = None
lower = float(exit_lower.iloc[i]) if
pd.notna(exit_lower.iloc[i]) else None

# 通道出场: 跌破下轨
if lower is not None and price_c < lower:
    should_exit = True
    exit_type = "channel"

# 止损出场
elif atr_val and price_c < stop_price:
    should_exit = True
    exit_type = "stop_loss"

if should_exit:

```

```

proceeds = total_shares * price_c * sell_factor
cash += proceeds

total_cost = sum(
    unit_shares * up * buy_factor for up in unit_prices
)
if total_cost > 0:
    trade_return = (proceeds - total_cost) / total_cost
else: trade_return = 0.0

trades.append({
    "buy_date": str(dates.iloc[0]).split(" ")[0] if
unit_prices else "",
    "sell_date": date_str,
    "sell_price": round(price_c, 2),
    "return": round(float(trade_return), 6),
    "win": trade_return > 0,
    "units": num_units,
    "exit_type": exit_type,
})

if exit_type == "channel":
    sell_signals.append({"date": date_str, "price":
round(price_c, 2)})
elif exit_type == "stop_loss":
    stop_signals.append({"date": date_str, "price":
round(price_c, 2)})

total_shares = 0.0
unit_prices = []
last_add_price = 0.0

# 当日组合净值
portfolio_values[i] = cash + total_shares * price_c

# 强制平仓
if total_shares > 0 and n > 0:
    last_price = float(close.iloc[-1])

```



```

last_date_str = str(dates.iloc[-1].date())
proceeds = total_shares * last_price * sell_factor
cash += proceeds

unit_shares_last = (initial_capital * RISK_PER_UNIT) /
float(atr.iloc[-1]) if pd.notna(atr.iloc[-1]) and float(atr.iloc[-1]) >
0 else 0
total_cost = 0
for up in unit_prices:
    if unit_shares_last > 0:
        total_cost += unit_shares_last * up * buy_factor
    else: pass

if total_cost > 0:
    trade_return = (proceeds - total_cost) / total_cost
elif unit_shares_last > 0 and unit_prices:
    total_cost = sum(unit_shares_last * up * buy_factor for up in
unit_prices)
    trade_return = (proceeds - total_cost) / total_cost if
total_cost > 0 else 0.0
else: trade_return = 0.0

trades.append({
    "buy_date": "",
    "sell_date": last_date_str,
    "sell_price": round(last_price, 2),
    "return": round(float(trade_return), 6),
    "win": trade_return > 0,
    "units": len(unit_prices),
    "exit_type": "force_close",
})
total_shares = 0.0
unit_prices = []
portfolio_values[-1] = cash

# 组合净值序列
pv_series = pd.Series(portfolio_values, index=dates)

```

```

# 基准（买入持有）
benchmark_shares = initial_capital / float(close.iloc[0]) if
float(close.iloc[0]) > 0 else 0
benchmark_values = close.values * benchmark_shares

# 回撤
running_max = np.maximum.accumulate(portfolio_values)
drawdown_pct = (portfolio_values / np.where(running_max > 0,
running_max, 1.0) - 1.0) * 100.0

# 每日收益率
daily_returns = pv_series.pct_change().dropna()

# 指标计算
total_days = (dates.iloc[-1] - dates.iloc[0]).days
n_years = total_days / 365.25 if total_days > 0 else 0.0
final_value = float(portfolio_values[-1])
total_return = (final_value / initial_capital - 1.0) * 100.0

if n_years > 0 and final_value > 0:
    annual_return = ((final_value / initial_capital) ** (1.0 / n_years)
- 1.0) * 100.0
else: annual_return = None

if len(daily_returns) > 1:
    std_ret = float(daily_returns.std())
    sharpe = float(daily_returns.mean() / max(std_ret, 1e-12) *
np.sqrt(252))
else: sharpe = None

max_dd = float(np.min(drawdown_pct))
max_dd_idx = int(np.argmax(drawdown_pct))
max_dd_date = str(dates.iloc[max_dd_idx].date())

if trades:
    wins = [t for t in trades if t["win"]]
    losses = [t for t in trades if not t["win"]]
    win_rate = len(wins) / len(trades) * 100.0

```

```

        avg_win = float(np.mean([t["return"] for t in wins])) if wins else
0.0
        avg_loss = abs(float(np.mean([t["return"] for t in losses]))) if
losses else 0.0
        pl_ratio = avg_win / avg_loss if avg_loss > 1e-12 else None
    else:
        win_rate = 0.0
        pl_ratio = None

benchmark_final = float(benchmark_values[-1])
benchmark_return = (benchmark_final / initial_capital - 1.0) * 100.0

# 序列化
def safe_list(series_like):
    if hasattr(series_like, "fillna"):
        return [round(float(x), 4) if pd.notna(x) and np.isfinite(x)
else None for x in series_like]
    return [round(float(x), 4) if (x is not None and pd.notna(x) and
np.isfinite(x)) else None for x in series_like]

dates_str = [str(d.date()) for d in dates]

return {
    "data": {
        "dates": dates_str,
        "close": safe_list(close),
        "entry_upper": safe_list(entry_upper),
        "exit_lower": safe_list(exit_lower),
        "stop_line": safe_list(stop_line),
        "buy_signals": buy_signals,
        "add_signals": add_signals,
        "sell_signals": sell_signals,
        "stop_signals": stop_signals,
        "portfolio_value": safe_list(pv_series),
        "benchmark_value": safe_list(benchmark_values),
        "drawdown": safe_list(drawdown_pct),
    },
    "metrics": {

```

```

        "total_return": round(total_return, 2),
        "annual_return": round(annual_return, 2) if annual_return is
not None else None,
        "sharpe_ratio": round(sharpe, 3) if sharpe is not None else
None,
        "max_drawdown": round(max_dd, 2),
        "max_drawdown_date": max_dd_date,
        "win_rate": round(win_rate, 1),
        "trade_count": len(trades),
        "profit_loss_ratio": round(pl_ratio, 2) if pl_ratio is not None
else None,
        "benchmark_return": round(benchmark_return, 2),
        "initial_capital": initial_capital,
        "final_value": round(final_value, 2),
        "n_days": n,
        "n_years": round(n_years, 2),
    },
}

```



图 1 海龟交易法则看板

任务 4. 通过调节策略的核心参数，如股票类型，通道周期等，观察收益情况的变化，总结海龟法则的适应场景和使用心得

1) 同一标的，不同通道周期

针对寒武纪标的，分别使用 20 日突破、10 日退出和 55 日突破、20 日退出两种组合回测 22-23 及 24-25 数据。

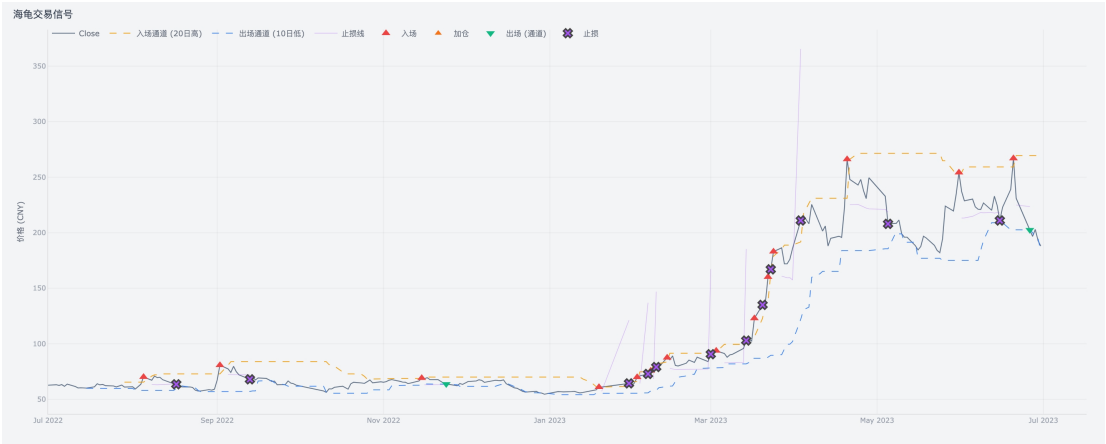


图 2 220701 至 230701 寒武纪(20,10)与(55,20)对比





图3 240701 至 250701 寒武纪(20,10)与(55,20)对比

在保持股票标的不变的情况下，对海龟交易策略采用不同的突破通道参数进行回测。通道周期越短，策略对价格变化越敏感，当股价突破历史高点时能够更快产生买入信号，因此交易次数明显增加；相反，通道周期越长，需要价格突破更长时间内的最高点才能触发建仓，策略对市场噪声具有更强的过滤能力，交易频率相对较低。

从本次实验结果来看，不同通道周期虽然导致了交易次数存在明显差异，但收益率之间并未呈现出稳定的大小规律，即通道周期更短或更长并不一定能够获得更高的收益。但的确可以观察到在市场存在持续且明确的趋势时，不同参数均有机会获得较好的收益，只是在入场时机和持仓周期上存在差异；而当市场处于震荡行情时，无论采用何种参数，都可能受到频繁假突破的影响。因此，参数优化只能在一定程度上改善策略表现，并不能从根本上改变策略的盈利能力。

2) 不同标的，相同策略

针对寒武纪和贵州茅台，使用 20 日突破、10 日退出策略回测 21-26 数据。图 4 上半部分为寒武纪数据，下半部分为贵州茅台数据。





图 4 20210701 至 20260701 不同类型标的收益对比

表 1 20210701 至 20260701 不同标的主要指标对比

标的	年化收益率 (%)	夏普比率	最大回撤 (%)	胜率 (%)
寒武纪	1.30	0.24	-16.26	70.80
贵州茅台	-1.39	-0.27	-15.97	51.70

在保持海龟交易策略参数不变的情况下，不同股票标的的回测结果存在较大差异。本次实验中，海龟策略在寒武纪上获得了较好的收益，而在贵州茅台上的收益则相对较差。这一结果说明，海龟交易策略的盈利能力与标的本身的价格运行特征密切相关。

海龟交易策略属于典型的趋势跟踪策略，最适用于具有持续上涨或持续下跌趋势的市场。当股票形成明显的单边行情时，策略能够通过突破信号及时进入市场，并利用较长时间持仓获取趋势带来的收益，实现“截断亏损、让利润奔跑”的交易思想。相反，对于长期处于震荡整理或趋势不连续的股票，价格容易频繁突破通道后又迅速回落，形成较多假突破，导致策略不断开仓和平仓，增加交易成本，并降低整体收益。因此，海龟交易策略的表现更多取决于市场是否具有持续趋势，而不是策略参数本身。

综合来看，海龟交易策略并非适用于所有股票，其优势主要体现在趋势性较强的标的和市场环境中。在实际投资过程中，应优先选择趋势明确、成交活跃、价格具有持续性的股票，同时结合市场整体走势或其他趋势确认指标，对震荡行情进行适当过滤，以进一步提高策略的稳定性和盈利能力。